Mardi 17/11, matin — Garder et trouver : filter et find

Kevin Bardoux

mardi 17 novembre 2026

Table des matières

Objectifs 2
Matin — filter et find 2
Rappel (5 min) 2
filter et find (15 min) 3
filter et find, en image 3
filter, case par case 4
find, case par case 4
filter et find, l'exemple minimal 4
filter et find — attention et vérification 5
Pas à pas ensemble — Les grands musées (35 min) 5
Variante à faire seuls 7
Enregistrer et envoyer (2 min) 8
Exercices — filter et find 8
Palier 1 — Les positifs, le premier mot long (35 min) 8
Palier 2 — Les mots longs, les prix TTC abordables (40 min) . 9
Palier 3 — filter et find à la main (35 min) 9
Enregistrer et envoyer (2 min) 10
À retenir 10
Si le temps le permet 10
Débrief 11
Sources 11

Note

Compétences visées : C4 · résolution de problème **Livrable** : le pas à pas `pas-a-pas-filter` envoyé sur GitHub, et `node --test l3-positifs.test.js` au vert.

! Le minimum du jour

- Le pas à pas « Les grands musées », fait avec le groupe.
- Le palier 1 : `node --test l3-positifs.test.js` et `node --test f1-premier-long.test.js` au vert.

Les paliers 2 et 3, puis la rubrique « Si le temps le permet », sont la suite naturelle, pour ceux qui avancent.

Objectifs

- Garder, dans un nouveau tableau, les éléments qui passent un test, avec `filter`.
- Trouver le premier élément qui passe un test, avec `find`, et savoir ce qu'il renvoie quand aucun ne convient.
- Choisir entre `map`, `filter` et `find`.

Comment se passe la matinée. Une seule notion : garder ou trouver des éléments d'un tableau. `filter` et `find` s'utilisent comme `map`, hier : on leur donne une fonction. La différence est dans ce qu'elles renvoient.

Matin — filter et find

Rappel (5 min)

1. Que renvoie `[1, 2].map((n) => n + 1)` ?
2. Quel score Lighthouse viser en accessibilité ?
3. Quelle commande lance les tests de `l1-doubler` ?

Trois questions courtes, sans regarder vos notes : la veille, la semaine passée, puis plus loin. Répondez sur une feuille ; la correction suit, à l'oral.

filter et find (15 min)

- **Pourquoi** : garder une partie d'un tableau, ou en trouver un élément.
- **`filter`** : un nouveau tableau, avec ceux qui passent le test.
- **`find`** : le premier qui passe le test, ou `undefined`.

Pourquoi. Les musées de plus de 50 000 visites ; le premier musée de Colmar dans une liste : on ne transforme pas les éléments, on en **garde** une partie, ou on en **cherche** un. Sans ces méthodes, il faut une boucle, une condition et un tableau à remplir.

Définition. La fonction donnée à `filter` ou à `find` est un test : elle renvoie vrai ou faux pour chaque élément. `filter` crée un **nouveau** tableau avec les éléments pour lesquels le test renvoie vrai (MDN Web Docs, s. d.-a). `find` renvoie le **premier** élément pour lequel le test renvoie vrai, ou `undefined` s'il n'y en a aucun (MDN Web Docs, s. d.-b). Aucune des deux ne modifie le tableau reçu.

filter et find, en image

Le même tableau, le même test (`price > 20`), trois méthodes : ce qui sort n'a pas la même forme.

`prices`	`map((price) => price > 20)`	`filter((price) => price > 20)`	`find((price) => price > 20)`
`10`	`false`	écarté	—
`25`	`true`	`25` gardé	`25`, et il s'arrête
`8`	`false`	écarté	(pas regardé)
`40`	`true`	`40` gardé	(pas regardé)
ce qui sort	**4 éléments**	**un tableau, `[25, 40]`**	**un élément, `25`**

`map` renvoie autant d'éléments qu'au départ ; `filter`, un tableau peut-être plus court, peut-être vide (`[]`) ; `find`, **un seul** élément, ou `undefined`.

filter, case par case

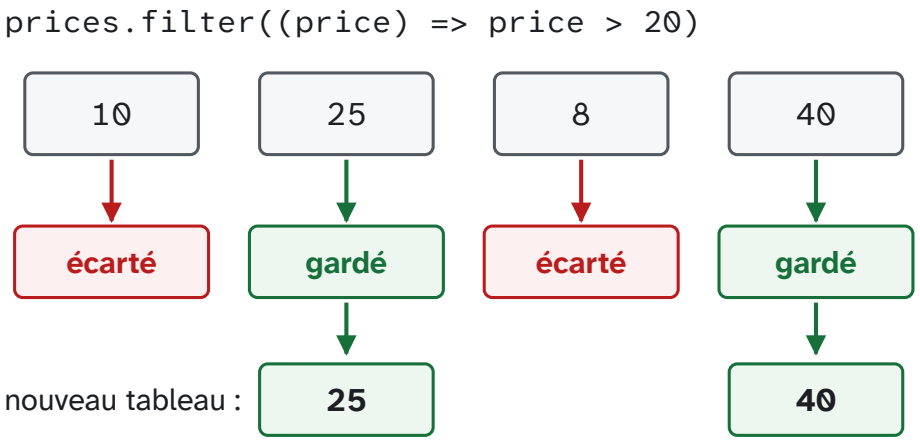


Figure 1. – filter garde les cases qui passent le test et écarte les autres.

find, case par case

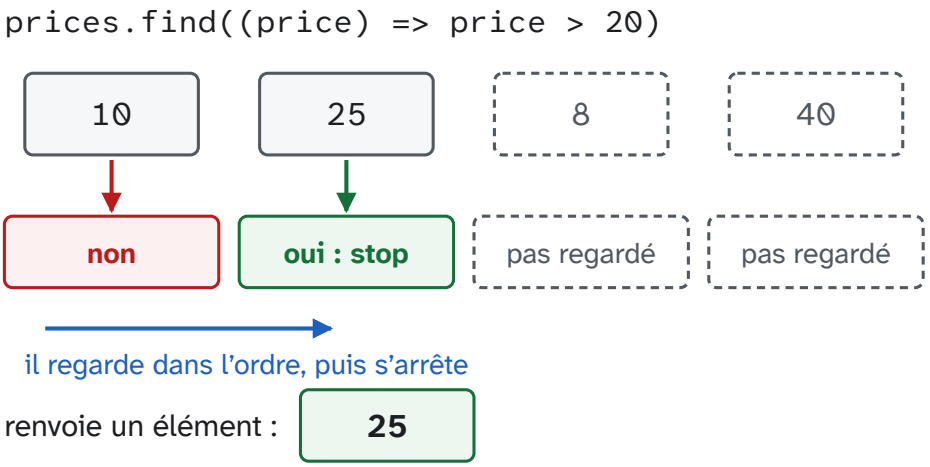


Figure 2. – find regarde les cases dans l'ordre et s'arrête à la première qui passe le test.

filter et find, l'exemple minimal

```
const prices = [10, 25, 8, 40];

const cheap = prices.filter((price) => price < 20);
// [ 10, 8 ] : un tableau

const firstExpensive = prices.find((price) => price >
20);
```

```
// 25 : un élément

const none = prices.find((price) => price > 100);
// undefined : aucun ne convient
```

filter et find — attention et vérification

> ⚠ Attention
>
> `find` ne renvoie pas un tableau : un élément, ou `undefined` si aucun ne convient. Avant de vous servir du résultat, vérifiez qu'il n'est pas `undefined`. `filter`, lui, renvoie toujours un tableau, parfois vide : `[]`.

- **Vérifiez** : que donne `[3, 8, 12].filter((n) => n > 5)` ?
- Et `[3, 8, 12].find((n) => n > 5)` ?

Pas à pas ensemble — Les grands musées (35 min)

- Le formateur code, vous tapez en même temps, une étape à la fois.
- Code et commentaires : `//` au-dessus de chaque ligne.
- Dossier : `pas-a-pas-filter`, fichier `visites.js`.
- Perdu ? Dossier `rattrapage/filter/`, une étape à la fois.

Palier 1 : guidé. Ouvrez `pas-a-pas-filter/visites.js` : il contient les visites de huit musées du Grand-Est en 2019 (Ministère de la Culture, s. d.). Dans le terminal : `cd pas-a-pas-filter`.

Avant de commencer. Lancez `node visites.js`, depuis `pas-a-pas-filter`. **Vous devez voir** : les données de départ, sans erreur. Chaque étape ajoute ses lignes à la fin du fichier, et sa sortie à la fin de la console.

Étape 1 — Garder les grands musées. À la fin du fichier, gardez les musées d'au moins 50 000 visites avec `filter`.

Avant d'exécuter : combien de nombres vont rester ?

```
// filter : garde les musées d'au moins 50 000 visites
const bigOnes = visits.filter((total) => total >=
```

```
50000);
// Affiche le nouveau tableau
console.log(bigOnes);
```

Enregistrez, puis lancez `node visites.js` dans le terminal, depuis `pas-a-pas-filter`.

Vous devez voir, à la fin de la sortie :

```
[ 51775, 59853, 165484, 179932 ]
```

Perdu ? Copiez `rattrapage/filter/etape-1/visites.js` à la place de votre `visites.js`, puis repartez avec le groupe.

Étape 2 — Compter, et vérifier l'original. Affichez le nombre de grands musées, puis `visits`.

Avant d'exécuter : `visits` a-t-il perdu des éléments ?

```
// length : combien de grands musées ?
console.log(bigOnes.length);
// Le tableau de départ n'a pas changé
console.log(visits);
```

Enregistrez, puis lancez `node visites.js` dans le terminal, depuis `pas-a-pas-filter`.

Vous devez voir, à la fin de la sortie :

```
4
[
    3165,  29212,
   51775,   3808,
    1207,  59853,
  165484, 179932
]
```

Perdu ? Copiez `rattrapage/filter/etape-2/visites.js` à la place de votre `visites.js`, puis repartez avec le groupe.

Étape 3 — Trouver le premier. Cherchez le premier musée de plus de 100 000 visites avec `find`.

Avant d'exécuter : `find` va-t-il renvoyer un tableau ou un nombre ?

```
// find : le premier musée de plus de 100 000 visites
const firstHuge = visits.find((total) => total > 100000);
// find renvoie UN élément, pas un tableau
console.log(firstHuge);
```

Enregistrez, puis lancez `node visites.js` dans le terminal, depuis `pas-a-pas-filter`.

Vous devez voir, à la fin de la sortie :

```
165484
```

Perdu ? Copiez `rattrapage/filter/etape-3/visites.js` à la place de votre `visites.js`, puis repartez avec le groupe.

Étape 4 — Quand rien ne convient. Cherchez un musée d'au moins un million de visites.

Avant d'exécuter : aucun musée n'atteint le million : que va renvoyer `find` ?

```
// find : aucun musée ne dépasse le million
const millionOrMore = visits.find((total) => total >= 1000000);
// Aucun élément trouvé : find renvoie undefined
console.log(millionOrMore);
```

Enregistrez, puis lancez `node visites.js` dans le terminal, depuis `pas-a-pas-filter`.

Vous devez voir, à la fin de la sortie :

```
undefined
```

Perdu ? Copiez `rattrapage/filter/etape-4/visites.js` à la place de votre `visites.js`, puis repartez avec le groupe.

Variante à faire seuls

Pour ceux qui ont validé l'étape avant les autres, dans le même fichier. Gardez les petits musées, ceux de moins de 5 000 visites, avec `filter`. Affichez-les. **Réussi si** : la console affiche `[ 3165, 3808, 1207 ]`.

Enregistrer et envoyer (2 min)

Depuis `~/dwwm/dwwm-tp/semaine-07` : `git status`, `git add pas-a-pas-filter`, `git commit -m "Ajoute le pas à pas filter et find"`, puis `git push`. **Réussi si** : `git status` affiche `rien à valider, la copie de travail est propre`.

Exercices — filter et find

Palier 1 — Les positifs, le premier mot long (35 min)

1. `l3-positifs.js` : garder les nombres positifs, avec `filter`.
2. `f1-premier-long.js` : le premier mot assez long, avec `find`.
3. **Réussi si** : leurs tests au vert.

Palier 1 : guidé. Durée : 35 min.

1. Ouvrez `l3-positifs.js`. `positives(numbers)` garde les nombres **strictement** positifs : `0` n'en fait pas partie.
2. Écrivez `return numbers.filter((n) => …);` : le test est « `n` est plus grand que `0` ».
3. Lancez `node --test l3-positifs.test.js`.
4. Ouvrez `f1-premier-long.js`. `firstLongWord(words, minLength)` renvoie le premier mot qui a **au moins** `minLength` lettres.
5. Écrivez `return words.find((word) => …);` : le test compare `word.length` et `minLength`.
6. Lancez `node --test f1-premier-long.test.js`. Puis un commit par exercice réussi.

Réussi si : `node --test l3-positifs.test.js` et `node --test f1-premier-long.test.js` affichent `ℹ fail 0`.

> ℹ Aides
>
> - **Le cours** : Mardi matin, « filter et find » ; le pas à pas, étapes 1 et 3.
> - **L'IA** : CS50.ai (palier IA 1) : demandez la différence entre « plus grand que » et « au moins », pas le code.
> - **MDN** : Array.prototype.filter() ; Array.prototype.find().

Palier 2 — Les mots longs, les prix TTC abordables (40 min)

1. `l4-mots-longs.js` : `filter`, avec un paramètre de plus.
2. `l6-prix-ttc.js` : `map`, puis `filter`, enchaînés.
3. **Réussi si** : leurs tests au vert.

Palier 2 : recommandé, semi-guidé. Durée : 40 min.

1. **`l4-mots-longs`** : `longWords(words, minLength)` garde **tous** les mots d'au moins `minLength` lettres. Même test que `f1`, autre méthode. `node --test l4-mots-longs.test.js`.
2. **`l6-prix-ttc`** : `affordablePricesWithTax(prices, max)` calcule le prix TTC de chaque prix (comme `l8`, hier), puis garde ceux qui sont inférieurs ou égaux à `max`.
   - Enchaînez : `prices.map(…).filter(…)`, une méthode par ligne.
   - Étape documentation : retrouvez dans MDN la fonction qui arrondit un nombre à l'entier le plus proche (page *Math*), pour arrondir au centime.
   - L'ordre compte : filtrez **après** le calcul du prix TTC.
   - `node --test l6-prix-ttc.test.js`.

Réussi si : `node --test l4-mots-longs.test.js` et `node --test l6-prix-ttc.test.js` affichent `ℹ fail 0`.

> ℹ Aides
>
> - **Le cours** : Mardi matin, « filter et find » ; Lundi après-midi, « Callback et map ».
> - **L'IA** : CS50.ai (palier IA 1) : demandez pourquoi l'ordre `map` puis `filter` change le résultat, pas le code.
> - **MDN** : Array.prototype.filter() ; Math.

Palier 3 — filter et find à la main (35 min)

- `myFilter` et `myFind`, sans les méthodes du même nom.
- **Réussi si** : leurs tests `.defi` au vert.

Palier 3 : autonome. Durée : 35 min.

Dans `tp-my-filter.js` et `tp-my-find.js`, écrivez `myFilter(array, callback)` et `myFind(array, callback)` : elles font ce que font `filter` et `find`, sans les utiliser. Pour `myFind`, réfléchissez : à quel moment la boucle peut-elle s'arrêter ?

Réussi si : `node --test tp-my-filter.defi.test.js` et `node --test tp-my-find.defi.test.js` affichent `ℹ fail 0`.

> ℹ Autonome
>
> Vous allez bloquer : c'est voulu, c'est le métier. Méthode : 1. lisez le message d'erreur en entier ; 2. cherchez-le dans la fiche « Lire un message d'erreur » ; 3. lisez la documentation ; 4. après 15 minutes bloqué sur le même point, demandez.

> ℹ Aides
>
> - **Le cours** : Mardi matin, « filter et find » ; votre `myMap` d'hier, s'il est fait.
> - **L'IA** : CS50.ai (palier IA 1) : demandez ce que fait `return` au milieu d'une boucle, pas le code.
> - **MDN** : Array.prototype.find(), section *Description*.

Enregistrer et envoyer (2 min)

Même rituel : `git status`, `git add`, `git commit -m "…"`, `git push`, depuis `semaine-07`.

À retenir

> À retenir
>
> - **`filter`** : un nouveau tableau, avec ceux qui passent le test ; parfois `[]`.
> - **`find`** : le premier qui passe le test, ou `undefined`.
> - **Le test** : une fonction qui renvoie vrai ou faux, sans parenthèses quand on la donne.

Si le temps le permet

- Les paliers 2 et 3 non terminés.
- Le parcours d'autonomie, lignes 4 et 6.

Débrief

- `map`, `filter`, `find` : laquelle pour « les noms de tous les musées » ?
- Et pour « le musée de Colmar » ?

Sources

MDN Web Docs. s. d.-a. « Array : méthode filter() - JavaScript ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/Array/filter.

MDN Web Docs. s. d.-b. « Array : méthode find() - JavaScript ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/Array/find.

Ministère de la Culture. s. d. « Fréquentation des Musées de France (data.gouv.fr) ». Consulté le 26 septembre 2026. https://www.data.gouv.fr/datasets/frequentation-des-musees-de-france-1.